DenguePharma

Relatório Consolidado de Implementação, Arquitetura e Evidências

Data de Geração: 05/11/2026 02:35 | **Versão de Entrega:** 0.45.0 | **Branch:** main (52+ commits sincronizados)

SUMÁRIO EXECUTIVO (VERSÃO 0.45.0): *Este documento consolida a totalidade dos componentes de software, infraestrutura multisserviço de contêineres, pipeline CI/CD GitHub Actions, controle de acesso formal RBAC para Leitor, Revisor, Adjudicador, Pesquisador e Administrador com 403 (REQ-GOV-001), trilha de auditoria tamper-evident com autor, instante ISO 8601 e valores antes/depois (REQ-GOV-003), conclusão integral de 100% de M08 (REQ-CLN-001..013), calibração clínica e síntese real documentada (REQ-CLN-006), conclusão integral de 100% de Relatórios e Painéis (M07 - REQ-REL-001..005), painéis por eixo (REQ-REL-003), metas não atingidas destacadas (REQ-REL-004), figuras acessíveis (REQ-REL-005), proveniência (REQ-REL-001), reprodutibilidade ReportSnapshot (REQ-REL-002), pseudonimização ética (REQ-GOV-002), esforço cognitivo (REQ-REV-005), conclusão de 100% de M05 (IA - 14 requisitos) e 100% de mensageria/operações (REQ-OPS-001..010), conformidade de licenças de IA (REQ-EXT-011), modo de ablação científica (REQ-EXT-012), backup e restauração testada (REQ-OPS-007), SLO operacional p95 <= 2.0 s (REQ-OPS-008), regenerabilidade canônica da camada ouro (REQ-OPS-009), contratos versionados (REQ-OPS-010), backoff com jitter e quarentena (REQ-OPS-005), governança orçamentária de IA (REQ-OPS-006), isolamento transacional SQL (REQ-OPS-003), concorrência por leases (REQ-OPS-004), resiliência Outbox/Inbox (REQ-OPS-001/002), idempotência por chave quíntupla (REQ-EXT-009), salvaguardas de tabelas/OCR (REQ-EXT-007), auditoria metrológica de IA (REQ-EXT-008), segurança contra injection (REQ-EXT-005), coerência numérica (REQ-EXT-006), auditoria de discrepâncias e outcome switching (REQ-ANA-005), versionamento por hash (REQ-ING-006), Grafo de Evidências Clínicas (REQ-ANA-008), cruzamento epidemiológico (REQ-ANA-004), PRISMA-ScR (REQ-SEL-004), meta-regressão (REQ-ANA-002/009), bioatividades ChEMBL, Carga Global de Doença (Eixo C), Kaplan–Meier (Eixo B), monitor de retratações (REQ-ANA-010), benchmark com Bootstrap (Eixo D - REQ-ANA-006/007) e salvaguardas RC-01 a RC-07. Total de* ***73 requisitos SDD formalmente verificados****.*

1. Resolução dos 10 Itens de Prioridade Alta (Auditoria Formal)

Em conformidade com a proposta v1.0 e os requisitos do PDF Analytics v2.0, foram executadas remediações atômicas e rastreadas na base de código:

Item / Requisito	Módulo	Solução Técnica Implementada e Validada
Item 1: CORS e Segurança Web	Spring Security	Configuração explícita de CORS em SecurityConfig.java autorizando origens locais seguras (localhost:5173, 3000, 8501). Testado em SecurityCorsTest.java.
Item 2: RFC 7807 Exception Handler	Backend REST	Tratamento unificado de exceções (MethodArgumentNotValid, AccessDenied, DataIntegrityViolation) gerando ProblemDetail padronizado. Testado em ApiExceptionHandlerTest.java.
Item 3: Integridade na Curadoria Humana	ExtractionService	Exigência compulsória de ROLE_ADJUDICADOR para campos críticos e bloqueio de criação de trechos sintéticos artificiais em correções humanas.
Item 4: Contrato de Revisão em Português	ReviewPage / API	Alinhamento das ações no frontend (ACEITAR, CORRIGIR, REJEITAR) com a máquina de estados do backend.
Item 5: Explorador de Alvos (Target Explorer)	Frontend / Rota	Criação de TargetExplorerPage.tsx estruturando NS5, NS3, proteínas estruturais e alvos do hospedeiro com sorótipos DENV 1-4.
Item 6: Estruturas Químicas 2D PubChem	Catálogo / UI	Integração dinâmica de imagens 2D via PubChem PUG REST nos cartões de substâncias catalogadas.
Item 7: Widget Monitor de Orçamento	Painel Operacional	Visualização em tempo real de limites e gastos de extração por IA em OperationalPanelPage.tsx.
Item 8: Conector ChEMBL REST API	Workers Python	Conector chembl.py para busca de moléculas, alvos virais de Flavivírus e bioatividades (IC50, EC50, Ki) com 5 testes pytest.
Item 9: Orquestração Docker Completa	Infraestrutura	Criação de Dockerfiles (backend Java multi-stage e frontend Nginx) integrados ao docker-compose.yml.
Item 10: Fallback sem Docker Daemon	Testes Backend	Adição de DockerClientFactory.isDockerAvailable() em AbstractIntegrationTest.java para execução ágil em qualquer host.

2. Expansões Estruturais e Científicas Avançadas (Eixos B, C, D e E)

A evolução do projeto integrou metodologias de ponta em química medicinal, epidemiologia e inteligência artificial:

Eixo / Requisito	Módulo e Componentes	Implementação, Contratos e Salvaguardas Metodológicas

Eixo D: Benchmark IA & Bootstrap REQ-ANA-006/007	Full-Stack Analytics benchmark.py ExtractionBenchmarkController ExtractionBenchmarkView.tsx	Avaliação frente a Gold Standard adjudicado: Precisão, Recall e F1 por campo/estrato, regras congeladas de sinônimos/números, segregação de ausências verdadeiras e Cluster Bootstrap de 95% (1000 réplicas por família). Exposição REST pública e painel visual interativo integrado no Dashboard.
Eixo B: Sobrevivência Kaplan–Meier REQ-ANA-003	Analytics + Frontend survival.py SurvivalReportingView.tsx	Curva S(t) de permanência sem relato com censura à direita na data de corte. Erro padrão de Greenwood, IC 95%, estratificação Indústria vs Academia e gráfico interativo em degraus no Frontend.
Monitor de Retratações REQ-ANA-010	Analytics Clinical retractions.py synthesis.py	Detecção de artigos retratados (Retraction Watch / Crossref) por DOI/PMID, expurgando-os automaticamente dos grupos de síntese clínica ativa e registrando motivo e data em tabela de auditoria.
Microdados Epidemiológicos REQ-ING-008	Workers Engine epidemiology.py	Ingestão de microdados OpenDengue e SINAN (DATASUS) com código ISO-3, ano, nível administrativo (UF/município), casos graves e óbitos com rastreabilidade SHA-256 e data de versão.
Carga Global (Eixo C)	Backend + Frontend /api/public/disease-burden DiseaseBurdenView.tsx	Mapeamento GBD/DATASUS e sinalização de **Desertos de Pesquisa** (Índia, Bangladesh, Indonésia) com hiperendemicidade e escassez de ensaios. Salvaguarda RC-04: não inventar zeros.
Shadow Screening por IA	Analytics Clinical shadow_screening.py	Modo sombra blinded dos revisores independentes; parecer consultivo exclusivo ao Adjudicador na divergência. Salvaguardas RC-01 a RC-07: soberania 100% humana na decisão.
Visualizador 3D PubChem	Frontend Target Explorer Molecule3DViewer.tsx	Conformações 3D interativas e alternância 2D SVG integradas aos alvos virais NS5, NS3, Proteína E e fatores do hospedeiro, exibindo massa molar e CID PubChem.
Bioatividades ChEMBL	Flyway V10 + Spring Boot /api/public/bioactivities	Tabela relacional substance_bioactivity registrando IC50, EC50 e Ki para inibidores canônicos (Balapiravir, Sofosbuvir, Cloroquina). Salvaguarda RC-02: segregação in vitro vs humano.
Pipeline CI/CD Contínuo	GitHub Actions .github/workflows/ci.yml	Validação contínua multisserviço em cada push cobrindo Java 21, spec/check.py, suítes pytest e testes de tipagem e build Vite do Frontend.

3. Topologia em Grafo de Rede de Evidências (Eixo E)

O Eixo E mapeia as interconexões farmacológicas complexas entre compostos candidatos, alvos biológicos (virais e do hospedeiro), desfechos observados e estudos primários:

- **Backend Spring Boot:** Implementação de EvidenceGraphController.java expondo GET /api/public/evidence-graph, com 20 nós tipados (SUBSTANCE, TARGET, OUTCOME, STUDY) e 13 arestas relacionais (INIBE, BLOQUEIA, MODULA, IMPACTA) vinculadas a níveis de evidência científica.
- **Frontend Interativo:** Página EvidenceGraphPage.tsx com renderização visual SVG em 4 colunas funcionais, drawer lateral com metadados do nó selecionado, filtros por categoria e legenda com distinção cromática por papel biológico.
- **Salvaguarda Científica RC-01 / RC-04:** Notificação compulsória no cabeçalho indicando que proximidade ou conectividade topológica em rede não constituem comprovação de eficácia clínica humana nem ranking.

4. Pipeline de Extração com LLM e Controle de Gastos (Workers)

Para possibilitar extrações com modelos de linguagem proprietários de forma controlada e auditável, foi desenvolvido o módulo denguepharma_workers.llm_adapter:

- **Suporte Multiprovedor:** Clientes para OpenAI Chat Completions (JSON mode), Google Gemini API (generateContent) e Mock determinístico para testes locais sem custo.
- **Medição e Tarifação em Tempo Real:** Medição exata de tokens de entrada (prompt) e saída (completion), calculando custos por chamada conforme tabela de preços USD.
- **Bloqueio Compulsório por Teto (BudgetGuard):** Interrupção imediata com BudgetExceededError caso o custo acumulado da execução ultrapasse o teto financeiro definido pelo pesquisador.
- **Abstenção Mandatória contra Alucinação (Cenário T4):** Toda saída de campo PRESENTE exige citação literal de trecho com verificação estrita no texto-fonte. Caso o modelo cite trecho inexistente, o status é automaticamente rebaixado para INCERTO e published=False.
- **Idempotência Reprodutível:** Chave SHA-256 única baseada nos 5 componentes fundamentais: document_version_id + pipeline_version + model_version + prompt_hash + execution_variant.

5. Métricas de Triagem Duplo-Cega: Concordância Inter-Avaliadores

Na Etapa 0.22.0 de Analytics, foram integradas formulações estatísticas rigorosas para calibração dos revisores independentes no processo de seleção PRISMA-ScR:

Métrica Estatística	Fórmula / Definição	Propósito Metodológico no DenguePharma
Concordância Observada (Po)	Po = Σ(diagonal) / Total	Proporção direta de votos idênticos entre os dois revisores independentes.
Kappa de Cohen (κ)	κ = (Po - Pe) / (1 - Pe)	Desconta a concordância esperada pelo mero acaso; inclui erro padrão assintótico SE(κ).
AC1 de Gwet	AC1 = (Po - Pe_gwet) / (1 - Pe_gwet)	Imune ao *paradoxo de prevalência* (Feinstein-Cicchetti), mantendo estabilidade quando há alta taxa natural de exclusão de artigos.
Matriz de Contingência 3x3	Pares [INCLUIR, EXCLUIR, INCERTO]	Mapeia detalhadamente concordâncias e divergências sem alterar decisões ou forçar resolução automática.

6. Orquestração e Validação Operacional em Contêineres Docker

A arquitetura completa do sistema foi unificada e validada via Docker Compose:

Serviço / Contêiner	Imagem Base	Portas	Papel Arquitetural e Estado Operacional
denguepharma-postgres	postgres:16-alpine	5434 -> 5432	Banco relacional com 10 migrações Flyway e triggers de imutabilidade.
denguepharma-rabbitmq	rabbitmq:3.13-management-alpine	5673, 15673	Fila durável de mensageria assíncrona com dead-letter exchange (DLQ).
denguepharma-minio	denguepharma-minio:local	9000, 9001	Armazenamento S3 imutável de documentos-fonte indexados por hash SHA-256.
denguepharma-backend	denguepharma-backend:latest	18080 -> 8080	API Spring Boot 3.4.5 Java 21 com segurança JWT, CORS e endpoints públicos/privados.
denguepharma-frontend	denguepharma-frontend:latest	5173 -> 80	Single Page Application (React/Vite/TypeScript) servida por Nginx Alpine.
denguepharma-worker	denguepharma-worker	Background	Processamento assíncrono de extração, idempotência e conectores científicos.

7. Resumo de Testes Automatizados e Rastreabilidade Completa

Todas as entregas foram estritamente validadas através de baterias automatizadas de testes:

- **Suíte Python Workers:** 55 testes pytest aprovados (`test_epidemiology`, `test_extractor`, `test_llm_adapter`, `test_chembl`, `test_pipeline`, `test_idempotency`).
- **Suíte Python Analytics:** 240 testes coletados (incluindo test_governance_rbac_and_audit com RBAC REQ-GOV-001 e auditoria REQ-GOV-003, test_pilot_synthesis_and_calibration com calibração e síntese real REQ-CLN-006 [100% M08], test_executive_panels REQ-REL-003..005 [100% M07], test_report_provenance_and_snapshot REQ-REL-001/002, test_review_telemetry REQ-GOV-002/REV-005, test_extraction_validation REQ-EXT-001..012, test_backup_and_performance REQ-OPS-007/008, test_reproducibility_and_contracts REQ-OPS-009/010, test_backoff_and_budget REQ-OPS-005/006, test_job_lease_concurrency REQ-OPS-003/004, test_transactional_messaging REQ-OPS-001/002, test_discrepancies, test_evidence_graph, test_trial_burden, test_selection_flow, test_meta_regression, test_benchmark, test_survival_reporting, test_retractions, test_shadow_screening, test_agreement).
- **Suíte Java Spring Boot:** Testes unitários e de integração aprovados (`ExtractionValidationUnitTest` com conformidade de licença e ablação, `PipelineIntegrationTest`, `TransactionalOutboxInboxUnitTest`, `DiscrepancyAuditControllerTest`, `EvidenceGraphControllerTest`, `TrialBurdenLinkControllerTest`, `PrismaFlowControllerTest`, `ExtractionBenchmarkControllerTest`, `BioactivityControllerTest`, `DiseaseBurdenControllerTest`).
- **Suíte Frontend TypeScript:** 31 testes Vitest aprovados em 7 suítes (`discrepancy-audit.test.tsx` com salvaguardas RC-01/RC-02, `evidence-graph.test.tsx`, `target-explorer.test.tsx`, `dashboard.test.tsx`, `curation-flows.test.tsx`).
- **Compilação Frontend:** Build de produção Vite gerado com sucesso sem erros ou pendências de tipos (60 módulos compilados).
- **Matriz de Engenharia:** 96 requisitos formais catalogados em spec/requirements.json e spec/rastreabilidade.csv, com **73 requisitos formalmente verificados** (incluindo REQ-GOV-001, REQ-GOV-003, REQ-CLN-006 [100% M08], REQ-REL-001..005 [100% M07], REQ-EXT-001..012, REQ-GOV-002, REQ-REV-005 [100% M05], REQ-OPS-001..010 [100% M07], REQ-ANA-005, REQ-ING-006, REQ-ANA-008, REQ-ANA-004, REQ-SEL-004, REQ-ANA-002 e REQ-ANA-009) e validados pelo script spec/check.py.